

Hardware VM Isolation in the Cloud

ENABLING
CONFIDENTIAL
COMPUTING WITH
AMD SEV-SNP
TECHNOLOGY

DAVID KAPLAN

When you rent an apartment, it is reasonable to have some expectation of privacy. The landlord provides you a place to stay in return for rent, and you don't expect the landlord to go snooping through your things, putting cameras in your bedroom, etc. When you rent a VM (virtual machine) in the cloud, why should expectations be any different?

Public clouds provide an important service, offering compute capability in return for money. While cloud providers arguably do their best to keep your data isolated, especially from other tenants, there typically isn't anything in hardware that actually keeps all your data isolated from the provider itself. Standard techniques such as network encryption and encrypted hard disks can protect data-in-transit and data-at-rest, respectively, but these don't protect data being actively processed (i.e., data-in-use). A bug in the cloud provider's software, a malicious insider, or an errant crash dump could all expose your private data in ways that are often undetectable. These risks have

kept many industries from moving to the cloud...until now, thanks to a new class of technology called “confidential computing.”

CONFIDENTIAL COMPUTING FOR VIRTUAL MACHINES

*Confidential computing*¹⁶ is a generic term for technology that is designed to protect workloads such as VMs from all other software, including the HV (hypervisor). It is a big change from the traditional trust model of CPUs. Historically, CPUs were built using a ring-based security model where higher-privileged rings had access to everything in the lower-privileged rings. This worked well for operating systems and many typical environments.

With confidential computing, this model is modified so that while administrator code, such as the HV, manages system resources and scheduling, that code no longer has complete access to the system. For example, the HV might choose whether to run one VM or another VM, and how much memory to assign each, but it cannot see or modify the contents of that memory.

Confidential computing is useful in environments where the owner of the workload differs from the owner of the physical machine. This may be because you're security-conscious or concerned about the risk of other tenants exploiting vulnerabilities in cloud infrastructure, or simply because of the type of data in the VM. Customer data, financial information, health information, etc., are highly sensitive and often subject to tight regulatory or compliance

controls. Confidential computing reduces the risk of that data leaking while in the cloud.

OVERVIEW OF AMD SEV-SNP

AMD began offering support for confidential computing with the first generation of SEV (Secure Encrypted Virtualization)¹² technology in 2017. The SEV technology has evolved over time to support more security and flexibility. This article focuses on the third generation of this technology, called SEV with Secure Nested Paging, or SEV-SNP. AMD SEV-SNP¹ is available in third-generation and later AMD EPYC processors, which began shipping in early 2021.

SEV-SNP leverages existing AMD-V technology, which provides special hardware support and acceleration for virtualization. SEV-SNP is designed to protect a VM from all outside entities, including the bare-metal hypervisor, BIOS (basic input/output system), other VMs, and even external I/O devices. While SEV-SNP requires special software support at the operating system and HV level, it does not require any changes to applications. A web server that runs in a cloud VM today can be easily moved into an SEV-SNP VM without recompilation in what is often referred to as “lift-and-shift.”

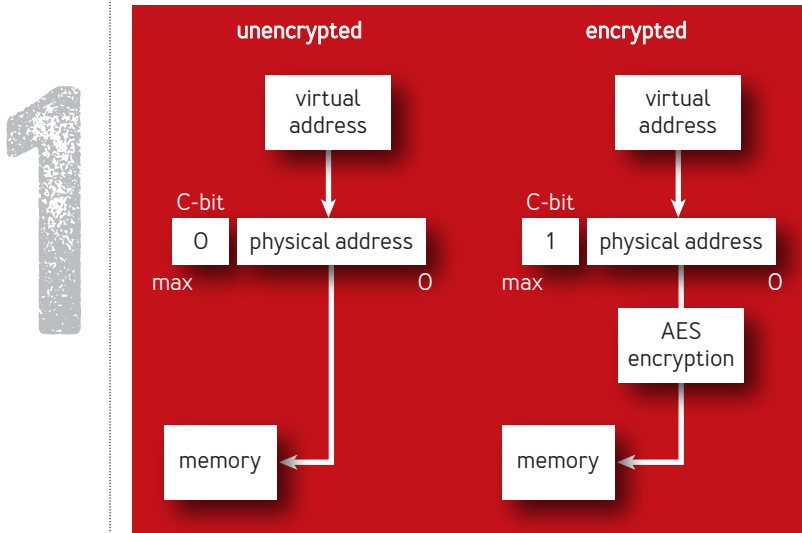
BASICS OF SEV-SNP

SEV-SNP supports several key principles of confidential computing, including the need for confidentiality, integrity, and attestation. The need for confidentiality should be obvious, as it's in the name *confidential* computing. In practical terms, this means restricting access to the data

inside an SEV-SNP VM. This is primarily accomplished using memory encryption, as shown in figure 1. AMD EPYC processors include a high-bandwidth AES-128 (or AES-256 in fourth-generation AMD EPYC) encryption engine in each of the on-die memory controllers. This engine is designed to quickly encrypt/decrypt all data between the CPU and DRAM (dynamic random access memory). Each SEV-SNP VM is assigned a unique random encryption key when the VM boots, and the use of that key is tightly controlled by hardware, ensuring that plaintext is visible to only the correct VM.

While most data inside a VM typically should remain private, VMs do sometimes need to communicate with the HV or devices. In the SEV-SNP architecture, this is done using *shared memory pages*. A VM may choose which

FIGURE 1: **MEMORY ENCRYPTION**



memory pages are private or shared using a page-table bit (C-bit or encrypted-bit). Private pages (C-bit=1) are always encrypted with the VM-specific encryption key, while shared pages (C-bit=0) may be either unencrypted or encrypted using an HV key.

Another key principle of SEV-SNP is data integrity, which means ensuring that untrusted code, like the HV, may not modify guest memory in any way. Even though guest memory is encrypted, an attacker could attempt to corrupt it or perform a replay attack, using an old copy of ciphertext to replace a newer version so the guest appears to see an old version of data. The SEV-SNP architecture enforces data integrity on guest private pages using a structure called the RMP (reverse map table).

REVERSE MAP TABLE

In an AMD system, memory is accessed through either CPU instructions or device I/O, known as DMA (direct memory access). CPU accesses use an internal MMU (memory management unit), while DMA uses an IOMMU (I/O memory management unit). Both the MMU and IOMMU translate virtual addresses into physical addresses using page tables before accessing memory. In the SEV-SNP architecture, the HV controls the page tables and mapping to addresses in the physical address space, but the RMP enforces access control in the physical address space.

The RMP is a large in-memory data structure that tracks the owner of each page of memory. Every 4KB page of DRAM is associated with a 16-byte RMP entry. That RMP entry indicates who is allowed to write to that page. At any given time, memory might be assigned to a specific

guest or to the HV, or even reserved for hardware use. The RMP is never modified by software directly; it can be manipulated only by trusted means such as special x86 CPU instructions.

The RMP is used to enforce access control to memory pages in the system. When the CPU or IOMMU translates a virtual address into a physical address, an RMP check is typically performed at the end of the translation. The physical address that the software or device wants to access is used to index into the RMP and then to verify that the assigned owner is the one trying to access the page. If this check fails, a fault is generated and access is blocked.

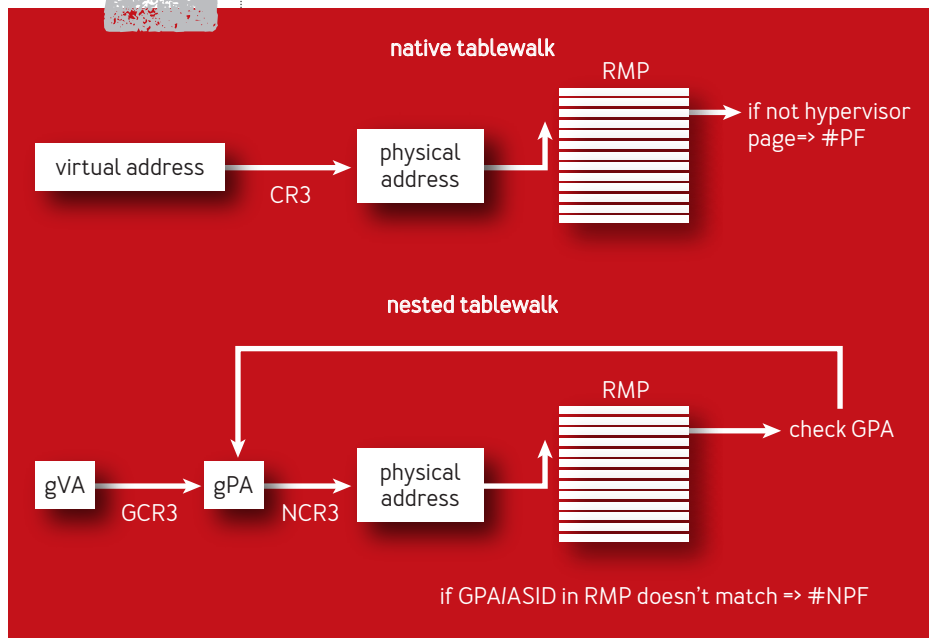
For example, to add a page of memory to a guest, the HV would issue the RMPUPDATE x86 instruction. This instruction modifies the RMP so that a specific page of memory is now assigned to a specific guest at a specific GPA (guest physical address). Once this is done, the HV no longer can write that memory page. If it attempts to do so, the CPU will issue a page fault.

What makes the RMP a reverse map is that for guest memory, the RMP entry contains the GPA and ASID (address space ID) to which that page is assigned. This ensures that a malicious HV cannot try to map a page into the wrong location in the guest address space.

When the CPU is running an SEV-SNP guest, after translating a GVA (guest virtual address) into a GPA and then into a system physical address using nested paging, the RMP entry is read and checked, as shown in figure 2. The RMP entry must contain the correct GPA that was used during the page tablewalk, which verifies that the translation is correct.

2

FIGURE 2: USING THE REVERSE MAP TABLE



OTHER PROTECTION

Memory encryption and the RMP provide a lot of confidentiality and integrity protection for SEV-SNP VMs, but this is not the whole story. SEV-SNP is designed to protect VMs from a malicious HV, malicious devices, etc. Of course, cloud providers probably aren't intending to be malicious, but the point of confidential computing is to be protected in all cases. After all, benign software can become malicious via a simple remote code exploit.

One example of additional security to protect against

potentially malicious hypervisors is the ability of SEV-SNP VMs to opt in to various modes of interrupt security. Typically, interrupts that occur for things like emulated devices are injected into the guest VM by the HV, and this is perfectly normal. But what would happen if the HV maliciously injected an interrupt while a VM had interrupts masked? Certainly, that's not expected behavior, but "unexpected behavior" is what makes attackers lick their chops.

One interrupt security mode offered by SEV-SNP is "Restricted Injection." In this mode, the HV is prevented from injecting any interrupts or exceptions into the guest other than one specific vector called #HV. When a #HV is injected, the guest uses a special protocol³ to find out what underlying interrupt or exception the hypervisor wanted to send. The guest may then decide to handle that event immediately or defer it until later. By doing so, the guest can effectively emulate its own interrupt controller, ensuring that interrupts are handled at the right time without relying on the HV.

Side-channel attacks such as Spectre are another potential threat that can break the isolation promises of technologies such as SEV-SNP. Consequently, SEV-SNP offers special modes, for example, to isolate the branch predictor so VMs can be protected from certain types of side-channel attacks.

Of course, confidential computing is not a panacea for all side-channel protection. Best practices such as constant-time crypto and avoiding secret-dependent branches are still required for the best level of security.

ATTESTATION

Technologies such as SEV-SNP can isolate a VM, ensuring data confidentiality and integrity... but how do you know that the cloud has actually turned the feature on? And how can you know if the cloud provider has installed the most recent firmware or microcode fixes on its system? This is where attestation comes into play.

Attestation is a fancy word for proof—it is proof that you are who you say you are, or that something is in fact what it looks like. It involves a trusted someone certifying (“attesting”) that someone else is secure.

In confidential compute technologies such as SEV-SNP, the cloud provider is untrusted, so the only entity that can attest to the security of a VM is the silicon provider itself. In SEV-SNP, this service is provided by an on-chip entity called the ASP (AMD security processor).

The ASP is a dedicated microcontroller that is part of the processor but separate from the x86 CPU. It has its own resources, including fuses, SRAM, crypto hardware, etc. The ASP runs firmware that AMD signs and provides a number of services to help manage the lifecycle of SEV-SNP VMs. These services are documented in a public specification⁴ and are used to create/destroy SEV-SNP VMs, perform tasks such as memory swapping or live migration, and—of course—to provide attestation.

In SEV-SNP, the initial code and data associated with a VM is not secret. To create a secure VM, the HV provides the ASP with this initial code and data, which is typically a guest BIOS image such as OVMF (open virtual machine firmware). Typically, a secure VM uses an encrypted hard disk, so in order to boot successfully, it will need to unlock

access to that disk. The key to this disk is likely on another key server somewhere. This is where attestation can come into play.

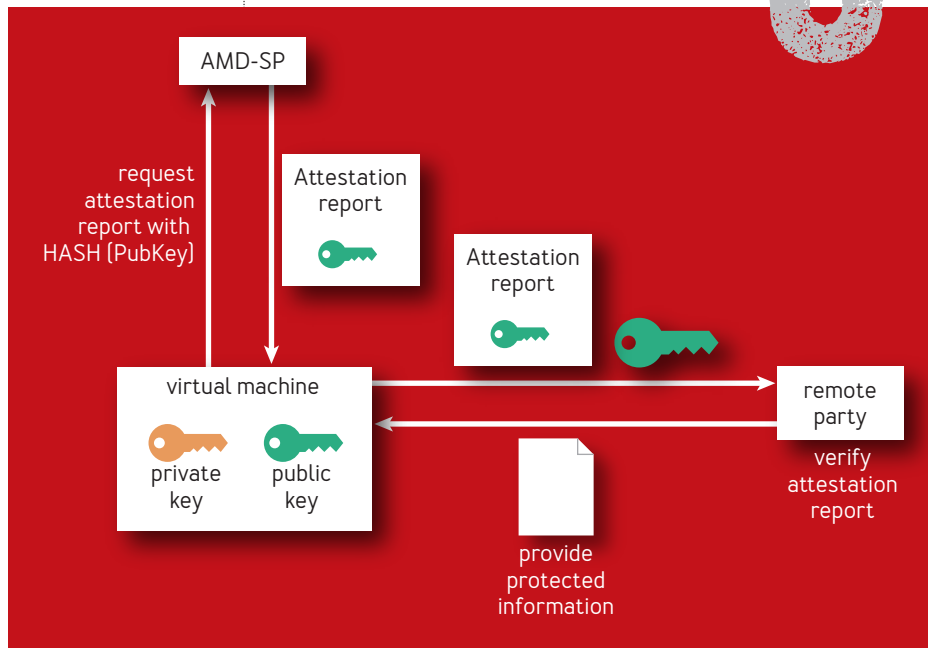
To request the disk decryption key, the new VM must prove itself to the key server. It must not only show that the VM is what it says it is, it must prove that it is running in a secure environment and that its image has not been tampered with in any way.

The ASP generates an attestation report when requested by an SEV-SNP guest VM, illustrated in figure 3. This attestation report includes lots of information about the VM and the platform itself, such as the measurement (i.e., hash) of the initial code and data of the VM, identity information about who the VM is, version numbers associated with the currently loaded firmware, etc.

The entire report is signed using a special key called a VCEK (versioned chip endorsement key). The VCEK is unique to each AMD CPU and is cryptographically derived using the current version of mutable ASP firmware and CPU microcode. This is built so someone looking at a report can be assured that this report was signed by a specific version of firmware, and not an older version trying to pretend to be a newer one. This is especially important since when researchers find bugs in features such as SEV-SNP, AMD will often issue a firmware or CPU microcode update to address them.

How does this all fit together to talk to a key server to do a secure VM boot? One of the fields in the attestation report is a 512-bit field containing arbitrary guest-supplied data. A typical use case might involve a guest VM that creates a public/private key pair and provides

3

FIGURE 3: **ATTESTATION REPORT CYCLE**

a hash of its public key to be included in the attestation report. The verifying party can then use that public key to communicate securely with the guest if it passes all the required security checks in the attestation report.

Besides verifying the data in the attestation report, the verifying party must check the signature of the report. It should be signed with the appropriate VCEK—but how does the verifier know it can trust that VCEK? To assist with this, AMD created the KDS (Key Distribution Service), which provides certificates that can prove a VCEK is authentic

and valid. The KDS is integrated into AMD's manufacturing flow so the AMD signs certificates only for authentic AMD parts. [For more details about this, see the VCEK Certificate and KDS Interface Specification.]⁵ An example tool that can be used to assist with SEV-SNP attestation can be found on GitHub.¹⁸

SECURITY WITHIN A VM

So far, this article has focused on protecting a VM from the hypervisor and other host software. But VMs often need to enforce their own internal security, too. For example, on a bare-metal system UEFI (Unified Extensible Firmware Interface) secure boot may use a TPM (trusted platform module) chip to calculate measurements of the boot flow to ensure protection from things like rootkits. In a virtualized system, a TPM can be emulated by the HV. How can this work in a confidential environment where the HV cannot be trusted to emulate a TPM?

To help with such a task, SEV-SNP provides a capability called VMPL (Virtual Machine Privilege Level), which allows for additional security controls that can protect memory inside the guest from other code within that same guest. Each SEV-SNP guest may have up to four VMPLs, where VMPLO is the most privileged and VMPL3 is the least. Each hardware context—called a VMSA (Virtual Machine Save Area)—is associated with a VMPL, and every memory page assigned to a guest may have different permissions based on VMPL. VMPLO always has full access to every page in the guest address space, but it may configure some pages not to be accessible at, say, VMPL1, or perhaps allow read-only access.

One use case of VMPLs is to run what AMD calls an SVSM [Secure VM Service Module] at VMPLO: a small piece of code designed to provide security services to the rest of the guest. For example, COCONUT⁸ is an SVSM written in Rust, which runs in VMPLO while the rest of the Linux OS runs at VMPL1.

One thing an SVSM can do is provide a vTPM (virtual-TPM) service to a guest. The vTPM appears as a normal TPM to the OS in the guest and enables features such as UEFI secure boot. But because the vTPM runs at VMPLO, it can use memory that is protected from the guest OS so any malware in the guest cannot compromise vTPM state. And let's not forget that the entire guest is running under SEV-SNP, so it's also protected from the HV or other VMs trying to compromise its state.

Services such as vTPM are also useful since they can prove the security of a VM to users who connect to that VM. The vTPM can attest to the security of the software running in the VM, while the SEV-SNP attestation report attests to the security of the vTPM itself and the VM configuration.

AMD is working with the open-source community to further SVSM development and eventually support features such as accelerated live migration and potentially other services via the SVSM.

Finally, the SEV-SNP architecture also supports the concept of a *paravisor*, which can assist with running a guest OS that knows almost nothing about SEV-SNP. This is a small but trusted layer that sits between a mostly unmodified legacy guest OS and the HV. The paravisor handles all the new security protocols associated with

SEV-SNP, even when the guest OS doesn't really know anything about confidential computing.

For example, Microsoft uses a custom paravisor in Azure that leverages SEV-SNP VMPL technology to support customers moving existing workloads into a confidential cloud with minimal effort.¹⁴ This paravisor supports compatibility with both Windows and Linux and provides secure boot services, secure interrupt delivery, and more. Technologies like this expand the reach of confidential computing by making security easier to adopt without requiring changes to customer software stacks.

COST OF SECURITY

Security never comes for free, but confidential compute technologies such as SEV-SNP try to minimize the overhead of security as much as possible. SEV-SNP uses high-bandwidth AES (Advanced Encryption Standard) memory-encryption engines, multilevel TLBs (translation lookaside buffers), and other microarchitectural optimizations to reduce the performance impact of the additional security checks required to maintain a confidential computing environment.

As a result, the performance cost of SEV-SNP is minimal in many workloads. For example, third-generation AMD EPYC processors demonstrated a delta of only approximately 4 percent on estimated SPECrate 2017_int_base,¹⁷ 2 percent on server-side Java performance benchmarks, and 2 percent on Black-Scholes benchmarks.⁹ These costs are primarily a result of the impact of AES memory encryption and RMP security checks. Over time, AMD expects these performance deltas will likely be

reduced further. Of course, actual performance of real-world workloads depends on many factors, including hardware configuration, placement of the VM and its resources, software optimizations, etc.

SECURING DEVICE I/O

With each generation of SEV, the technology has evolved to add more security features, capabilities, and improved performance. AMD recently announced the next major evolution for SEV with the new SEV-TIO (SEV with Trusted I/O) technology.² SEV-TIO provides support for direct device assignment to SEV-SNP VMs, allowing secure VMs and physical devices to communicate in a fast and secure manner.

SEV-TIO is built atop several industry standards, including PCIe's TDISP (TEE Device Interface Security Protocol),¹⁵ IDE (Integrity and Data Encryption),¹¹ SPDm (Secure Protocol and Data Model),¹⁰ and others. It provides a secure link between a device, such as a virtual function on a device, and a specific SEV-SNP VM.

This secure link means that communication between the VM and the device has both confidentiality and integrity. It provides a way for a device to attest to a guest VM, so guest VMs can avoid interacting with malicious devices. Most importantly, it enables high-performance DMA without bounce buffering.

Prior to AMD SEV-TIO, all device communication had to go through shared (unencrypted) memory pages. This meant that a VM would have to take the data it wanted to send to a device, copy it into a C-bit=0 page, and then ask the device to grab it from there. That extra memory copy

can be pretty expensive if you're doing a lot of I/O. With SEV-TIO, once the guest has verified the attestation of the device and decided it is trustworthy, it can allow the device to DMA directly to guest private (encrypted) memory; no more memory copies required.

Additionally, because any data in C-bit=0 pages is visible to the HV, any secrets the VM wants to send to a device must be encrypted in software before being sent over DMA. With SEV-TIO, this software encryption is no longer strictly required as the data being read and written by devices can target C-bit=1 pages.

SEV-TIO is designed to work with any device that is compliant with TDISP and related standards. This might include SmartNICs (smart network interface cards), which can offload network and storage work; AI training accelerators; and more. While not every workload may require the kind of I/O security and performance offered by direct device assignment, SEV-TIO expands the reach of confidential computing to I/O-sensitive workloads.

CONCLUSION

Confidential computing is a security model that fits well with the public cloud. It enables customers to rent VMs while enjoying hardware-based isolation that ensures that a cloud provider cannot purposefully or accidentally see or corrupt their data. SEV-SNP was the first commercially available x86 technology to offer VM isolation—including confidentiality and integrity—for the cloud and is deployed in Microsoft Azure,⁷ AWS,⁶ and Google Cloud.¹⁹ As confidential computing technologies such as SEV-SNP

develop, confidential computing is likely to simply become the default trust model for the cloud.

References

1. AMD. 2020. AMD SEV-SNP: Strengthening VM isolation with integrity protection and more. AMD White Paper; <https://www.amd.com/system/files/TechDocs/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf>.
2. AMD. 2023. AMD SEV-TIO: trusted I/O for secure encrypted virtualization. AMD White Paper; <https://www.amd.com/content/dam/amd/en/documents/developer/sev-tio-whitepaper.pdf>.
3. AMD. 2023. SEV-ES guest hypervisor communication block standardization; <https://www.amd.com/system/files/TechDocs/56421-guest-hypervisor-communication-block-standardization.pdf>.
4. AMD. 2022. SEV secure nested paging firmware ABI specification; <https://www.amd.com/system/files/TechDocs/56860.pdf>.
5. AMD. 2023. Versioned chip endorsement key (VCEK) certificate and KDS interface specification; <https://www.amd.com/system/files/TechDocs/57230.pdf>.
6. AWS. 2023. Amazon EC2 now supports AMD SEV-SNP; <https://aws.amazon.com/about-aws/whats-new/2023/04/amazon-ec2-amd-sev-snp/>.
7. Cai, R. 2022. Azure confidential VMs using SEV-SNP [DCasv5/ECasv5] are now generally available. Microsoft Azure Confidential Computing Blog; <https://techcommunity.microsoft.com/t5/azure-confidential-computing/azure-confidential-vm-using-sev-snp->

- dcasv5-ecav5-are-now/ba-p/3573747.
8. COCONUT Secure VM Service Module; <https://github.com/coconut-svsm/svsm>.
 9. Comp, L. 2021. Microsoft Azure confidential computing powered by 3rd gen EPYC CPUs. AMD; <https://community.amd.com/t5/epyc-processors/microsoft-azure-confidential-computing-powered-by-3rd-gen-epyc/ba-p/497796>.
 10. DMTF. 2023. All published versions of DSP0274; <https://www.dmtf.org/dsp/DSP0274>.
 11. Harriman, D. 2022. Integrity and Data Encryption and IO security updates. PCI-SIG; <https://pcsig.com/blog/integrity-and-data-encryption-ide-and-io-security-updates>.
 12. Kaplan, D., Powell, J., Woller, T. 2021. AMD memory encryption. AMD White Paper; <https://www.amd.com/system/files/TechDocs/memory-encryption-white-paper.pdf>.
 13. Linux SVSM; <https://github.com/AMDESE/linux-svsm>.
 14. Perez-Vargas, C. 2023. Confidential VMs on Azure. Microsoft Windows OS Platform Blog; <https://techcommunity.microsoft.com/t5/windows-os-platform-blog/confidential-vms-on-azure/ba-p/3836282>.
 15. PCI-SIG. 2022. TEE Device Interface Security Protocol; <https://pcsig.com/tee-device-interface-security-protocol-tdisp>.
 16. Russinovich, M., et al. 2021. Toward confidential cloud computing. *Communications of the ACM* 64 (6), 54–61; <https://dl.acm.org/doi/10.1145/3453930>.
 17. See <https://www.spec.org> for more information about SPEC® benchmarks.

18. VirTEE; <https://github.com/virtee/>.
19. Young, J. 2023. Oh SNP! VMs get even more confidential. Google Cloud Blog; <https://cloud.google.com/blog/products/identity-security/rsa-snp-vm-more-confidential>.

David Kaplan *is a senior fellow at AMD, where he is the lead architect for the AMD Secure Encrypted Virtualization technology as part of the AMD Product Security Office. Kaplan has more than 17 years of experience at AMD and has worked on confidential computing for the past 10 years. He has filed more than 50 patents in his career so far and has a B.S. from the University of Illinois.*

Copyright © 2023 held by owner/author. Publication rights licensed to ACM.

SHAPE THE FUTURE OF COMPUTING!

Join ACM today at acm.org/join

**BE CREATIVE. STAY CONNECTED.
KEEP INVENTING.**



Association for
Computing Machinery

Advancing Computing as a Science & Profession